Занятие 46.3. Ссылки и const

Так же, как можно объявить **указатель на константное значение**, так же можно объявить и **ссылку** на константное значение в языке C++.

Оглавление:

1. Ссылки на константные значения
2. Инициализация ссылок на константы
3. Ссылки r-values
4. Константные ссылки в качестве параметров функции

Ссылки на константные значения

Объявить ссылку на константное значение можно путем добавления **ключевого слова const перед типом данных**:

```
const int value = 7;
const int &ref = value; // ref - это ссылка на константную переменную value
```

Ссылки на константные значения часто называют просто ***«ссылки на константы»*** или «***константные ссылки***».

Инициализация ссылок на константы

В отличие от ссылок на неконстантные значения, которые могут быть инициализированы только неконстантными **l-values**, ссылки на константные значения могут быть инициализированы неконстантными l-values, константными l-values и r-values:

```
int a = 7;
const int &ref1 = a; // ок: a - это неконстантное l-value

const int b = 9;
const int &ref2 = b; // ок: b - это константное l-value

const int &ref3 = 5; // ок: 5 - это r-value
```

Как и в случае с указателями, константные ссылки также могут ссылаться и на неконстантные переменные. При доступе к значению через константную ссылку, это значение автоматически считается const, даже если исходная переменная таковой не является:

```
int value = 7;
const int &ref = value; // создаем константную ссылку на переменную value

value = 8; // ок: value - это не константа
ref = 9; // нельзя: ref - это константа
```

Ссылки r-values

Обычно r-values имеют область видимости выражения, что означает, что они уничтожаются в конце выражения, в котором созданы:

```
std::cout << 3 + 4; // 3 + 4 вычисляется в r-value 7, которое уничтожается в конце этого стейтмента
```

Однако, когда константная ссылка инициализируется значением r-value, время жизни r-value продлевается в соответствии со временем жизни ссылки:

```
int somefcn()
{
    const int &ref = 3 + 4; // обычно результат 3 + 4 имеет область видимости выражения и уничтожился бы в конце этого стейтмен
    std::cout << ref; // мы можем использовать его здесь
} // и время жизни r-value продлевается до этой точки, когда константная ссылка уничтожается
```

Константные ссылки в качестве параметров функции

Ссылки, используемые в качестве параметров функции, также могут быть константными. Это позволяет получить доступ к аргументу без его копирования, гарантируя, что функция не изменит значение, на которое ссылается ссылка:

```
// ref - это константная ссылка на переданный аргумент, а не копия аргумента
void changeN(const int &ref)
{
    ref = 8; // нельзя: ref - это константа
}
```

Ссылки на константные значения особенно полезны в качестве параметров функции из-за их универсальности. Константная ссылка в качестве параметра позволяет передавать неконстантный аргумент l-value, константный аргумент l-value, **литерал** или результат выражения:

```
#include <iostream>

void printIt(const int &a)
{
    std::cout << a;
}

int main()
{
    int x = 3;
    printIt(x); // неконстантное l-value

    const int y = 4;
    printIt(y); // константное l-value

    printIt(5); // литерал в качестве r-value

    printIt(3+y); // выражение в качестве r-value

    return 0;
}
```

Результат выполнения программы:

```
3457
```

Во избежание ненужного, слишком затратного копирования аргументов, переменные, которые не являются фундаментальных типов данных (типов int, double и т.д.) или указателями, — должны передаваться по (константной) ссылке в функцию. Фундаментальные типы данных должны передаваться по значению в случае, если функция не будет изменять их значений.

Правило: Переменные нефундаментальных типов данных и не являющиеся указателями, передавайте в функцию по (константной) ссылке.